

Nombre JavaScript

Dans ce chapitre, vous apprendrez à représenter des nombres en JavaScript.

Rappels

JavaScript prend en charge les nombres entiers et à virgule flottante qui peuvent être représentés en notation décimale, hexadécimale ou octale. Contrairement à d'autres langages, JavaScript ne traite pas différemment les nombres entiers et à virgule flottante. Tous les nombres en JavaScript sont représentés sous forme de nombres à virgule flottante. Voici un exemple démontrant les nombres dans différents formats

```
let x = 2;    // integer number  
let y = 3.14; // floating-point number  
let z = 0xff; // hexadecimal number
```

Rappel: Opération sur des nombres et des chaînes

Comme vous le savez dans les chapitres précédents, l'opérateur **+** est utilisé à la fois pour l'addition et la concaténation. Ainsi, effectuer des opérations mathématiques sur des nombres et des chaînes peut produire des résultats intéressants. L'exemple suivant vous montrera ce qui se passe lorsque vous ajoutez des nombres et des chaînes :

```
var x = 10;  
var y = 20;  
var z = "30";  
document.write(x + y); // 30  
document.write("<br>");
```

Si vous observez attentivement l'exemple ci-dessus, vous constaterez que le résultat de la dernière opération n'est pas simplement une simple concaténation de chaînes, car les opérateurs ayant la même priorité sont évalués de gauche à droite . C'est pourquoi, puisque les variables x et y sont toutes deux des nombres, elles sont ajoutées en premier, puis le résultat est concaténé avec la variable z qui est une chaîne, d'où le résultat final est $30 + "30" = "3030"$.

Analyser des entiers à partir de chaînes

La méthode `parseInt()` peut être utilisée pour analyser un entier à partir d'une chaîne. Cette méthode est particulièrement pratique dans les situations où vous avez affaire à des valeurs telles que des unités CSS, par exemple 50px, 12pt, etc. et que vous souhaitez en extraire la valeur numérique.

Si la méthode `parseInt()` rencontre un caractère qui n'est pas numérique dans la base spécifiée, elle arrête l'analyse et renvoie la valeur entière analysée jusqu'à ce point. Si le premier caractère ne peut pas être converti en nombre, la méthode retournera NaN(not a number).

Les espaces de début et de fin sont autorisés. Voici un exemple

```
document.write(parseInt("3.14") + "<br>"); // 3
document.write(parseInt("50px") + "<br>"); // 50
document.write(parseInt("12pt") + "<br>"); // 12
document.write(parseInt("0xFF", 16) + "<br>"); // 255
document.write(parseInt("20 years") + "<br>"); // 20
document.write(parseInt("Year 2048") + "<br>"); // NaN
document.write(parseInt("10 12 2018")); // 10
```

De même, vous pouvez utiliser la méthode `parseFloat()` pour analyser un nombre à virgule flottante à partir d'une chaîne. La méthode `parseFloat()` fonctionne de la même manière que la méthode `parseInt()`, sauf qu'elle récupère à la fois des entiers et des nombres avec des décimales.

```
document.write(parseFloat("3.14") + "<br>"); // 3.14
document.write(parseFloat("50px") + "<br>"); // 50
document.write(parseFloat("1.6em") + "<br>"); // 1.6
document.write(parseFloat("124.5 lbs") + "<br>"); // 124.5
document.write(parseFloat("weight 124.5 lbs") + "<br>"); // NaN
document.write(parseFloat("6.5 acres")); // 6.5
```

Conversion de nombres en chaînes

La méthode `toString()` peut être utilisée pour convertir un nombre en son équivalent de chaîne. Voici un exemple:

```
var x = 10;  
var y = x.toString();  
document.write(y); // '10'  
document.write(typeof y + "<br>"); // string  
document.write(typeof x + "<br>"); // number  
  
document.write((12).toString() + "<br>"); // '12'  
document.write((15.6).toString() + "<br>"); // '15.6'  
document.write((6).toString(2) + "<br>"); // '110'
```